
그래프 데이터베이스 개요와 Neo4j
{ Vector • Graph 데이터베이스 }

GRAPH DATABASE · NEO4J

그래프 데이터베이스와 Neo4j

연결을 저장하고, 따라가며 탐색하는 데이터베이스



⚠ 관계형 DB : 깊어질수록 JOIN 중첩 → 비용 증가

1 **왜 그래프 DB인가**
개체에서 연결로, 집계형에서 탐색형으로

2 **핵심 원리**
Index-free Adjacency

3 **Neo4j의 구조**
특징 · 아키텍처 · 에디션

4 **실습 환경과 첫 한글 데이터**
Desktop · Docker · AuraDB · Cypher

5 **적용 기준과 도입 리스크**
대체가 아니라 병용

I · 데이터 패러다임의 변화

개체에서 연결로

~~개체 (Entity)~~ → **연결 (Connection)** 데이터의 가치가 이동한다

추천	무엇을 샀는가	→	누가 · 무엇과 · 어떻게 연결되어 있는가
친구 추천	친구가 몇 명인가	→	누구를 거쳐 누구와 이어지는가
AI · RAG	문서의 유사도	→	사실 간의 의미 관계 · 근거 경로

답은 개체가 아니라 연결에 있다

I · 데이터 패러다임의 변화

집계형에서 탐색형으로

집계형 질의 → **탐색형 질의** 질의 요구의 이동

집계형

"이번 달 **매출 합계**는 얼마인가?"



Σ 합계

관계형 DB에 유리 많은 행을 모아 계산

탐색형

"내 **친구의 친구** 중에 **같은 취미**를 가진 사람은?"



그래프 DB에 유리 연결을 따라간다

◎ **어떤 질문을 많이 던지느냐** 가 DB를 고르는 첫 기준

관계형 DB의 한계 ① JOIN 중첩

관계 = 외래키(FK) 값으로만 표현

사람		모임
id · 이름 · 모임_id	FK 값 일치 ↔	id · 이름 · 장소

관계는 저장되지 않고 **질의 시점에 JOIN으로 계산**

T1 JOIN T2 JOIN T3 JOIN T4 JOIN T5

1단계 관계
JOIN 1회

4단계 관계
JOIN 4회 이상 + 중간 결과 폭증

4단계 추적 질의를 SQL로 쓸 수 있을까?

탐색 깊이에 따른 응답 시간



관계형 DB의 한계 ② 스키마 부담

관계형 DB : 테이블 + 연결 테이블

⚠ N:M 마다 연결 테이블 → 스키마 복잡도 ↑

컬럼·관계 추가 = DDL + 마이그레이션

사람
id · 이름
1 · 김민수
2 · 이서연

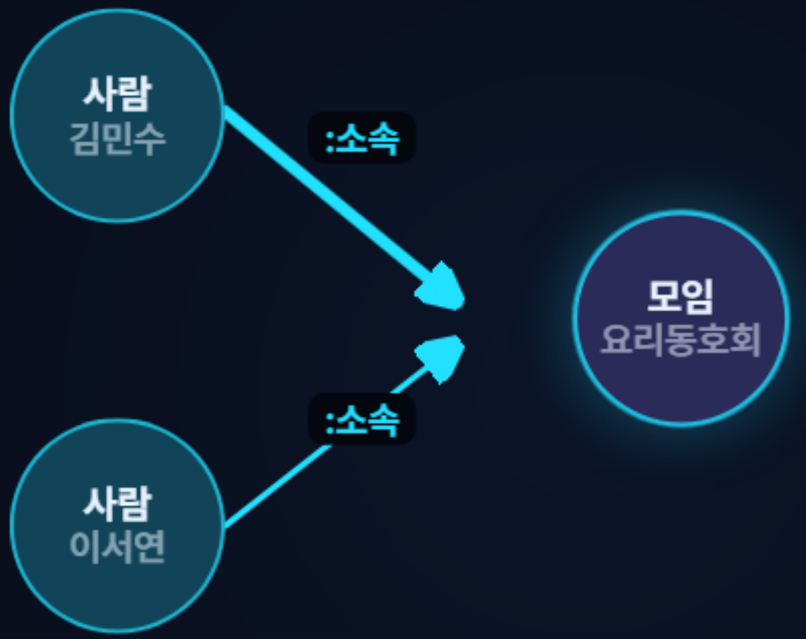
사람_모임 (연결)
사람_id · 모임_id
1 → 10
2 → 10

모임
id · 이름
10 · 요리동호회

관계는 키 값을 맞춰 봐야 보인다

```
사람.id = 사람_모임.사람_id AND …
```

그래프 DB : 노드 + 관계



관계가 **선 하나**로 바로 보인다

[그림 1-2] 관계형 DB와 그래프 DB의 데이터 표현 방식 비교

III · 그래프 데이터베이스의 정의

그래프 데이터베이스란

노드 = 개체



관계 = 1급 시민 · 저장 대상

타입 :소속 **방향** 사람 → 모임 **속성** {가입일}

그래프 DB = 데이터를 **노드(Node)**와 **관계(Relationship)**로 저장·탐색하는 DBMS

질의 = 패턴 매칭

```
MATCH (a:사람)-[:소속]->(m:모임)
RETURN a, m
```

찾고 싶은 그래프 모양을 그리면 그대로 검색

~~값이 같은 것을 찾는 도구~~ → **어떻게 연결되어 있는지 따라가는 도구**

IV · 주요 활용 분야

주요 활용 분야 ①

금융

- 이상거래탐지 FDS
- 자금세탁방지 AML
  자금 흐름 추적
- 신용 연계 분석

소셜 · 추천

- 친구 추천
- 협업 필터링
- 영향력 있는 사용자 식별

IT 운영

- IT 자산 · 서비스 의존성 관리
- 장애 영향도 분석

금융 · 소셜 · IT 운영의 **핵심 업무**에서 이미 그래프가 쓰인다

IV · 주요 활용 분야

주요 활용 분야 ②

금융 | 소셜 · 추천 | IT 운영 | + 3

보안

- 권한 상속 경로 분석
- 침해 경로 탐지

공공 · 행정

- 인허가 절차 흐름
- 법령 · 지침 간 참조 관계 관리

AI

- 지식그래프 기반 RAG
- 개체 연결
- 근거 기반 추론

공통점 : 무엇이 무엇과 어떻게 이어지는가를 묻는 업무

V · INDEX-FREE ADJACENCY

Index-free Adjacency ① 원리

Index-free Adjacency = 인덱스 없는 인접성

노드 레코드 : 나

첫 관계 포인터	→ #12
다음 관계	→ #13
다음 관계	→ #14



탐색 비용

∝ **실제로 방문한 이웃 수**

~~전체 데이터 크기~~ 와는 무관

네이티브 그래프 DB

각 노드가 인접 관계의 **물리적 위치(포인터)**를 직접 보유

이웃 탐색 = 포인터를 따라 이동

~~인덱스 조회~~ → 포인터로 곧바로 이동

전체를 뒤지지 않고 이웃만 따라간다

V · INDEX-FREE ADJACENCY

Index-free Adjacency ② 비용 비교

데이터 크기에 따른 탐색 비용



⚠ 단, 주의

탐색을 시작할 노드 = **앵커(Anchor)**
앵커를 찾을 때는 **인덱스가 필요**

인덱스 → 시작 노드 → 포인터 탐색

인덱스 설계는 **여전히 중요**

깊은 관계 탐색에서 **성능 차이**를 만드는 원리

VI · 그래프 데이터 모델의 두 갈래

LPG와 RDF ① 단위 · 속성 · 질의 언어

	속성 그래프 LPG	RDF 시맨틱 그래프
기본 단위	노드 — 관계 — 노드 + 속성	주어 → 술어 → 목적어 트리플
속성 표현	노드 · 관계에 직접 키-값 {이름:'김민수'}	별도 트리플로 표현 (김민수, 이름, '김민수')
질의 언어	Cypher · Gremlin · GQL	SPARQL

VI · 그래프 데이터 모델의 두 갈래

LPG와 RDF ② 표준화와 강점

	속성 그래프 LPG	RDF 시맨틱 그래프
표준화	**GQL 국제 표준** ISO/IEC 39075 ✓ 국제 표준화 완료	**W3C 표준** RDF · RDFS · OWL 웹 표준 어휘 체계
강점	✓ 개발 친화적 ✓ 모델링이 직관적 ✓ 성능 튜닝 용이	✓ 의미 추론 ✓ 조직 간 데이터 통합 · 공유

◎ 두 모델은 **지향하는 방향**이 다르다 : 개발 생산성 vs 의미 공유

VI · 그래프 데이터 모델의 두 갈래

대표 제품과 Neo4j의 선택

LPG 계열

- Neo4j
- Amazon Neptune
- TigerGraph

Neo4j = LPG 모델 채택
개발 생산성 · 탐색 성능에 강점

RDF 계열

- GraphDB
- Stardog
- AllegroGraph

RDF = 온톨로지 기반 추론
표준 어휘 공유에 강점

두 모델을 병행하는 사례도 증가

LPG : 서비스 · 탐색 + RDF : 추론 · 표준 어휘 공유 각자의 강점을 살려 함께 쓴다

VI · 그래프 DB의 종류와 특성

그래프 DB의 종류 ① 저장 구조에 따라

1 네이티브 그래프 DB

노드·관계를 **그래프 전용 구조**로 직접 저장 → 깊은 탐색에 유리

- Neo4j
- Memgraph
- TigerGraph

2 멀티모델 DB

문서·키 값 저장소 위에 **그래프 기능을 함께** 제공

- ArangoDB
- OrientDB
- Azure Cosmos DB

3 분산 저장소 위의 그래프 계층

기존 **분산 저장소** 위에 그래프 계층을 얹음 → 대규모 확장

- JanusGraph

위에서 동작: **Cassandra** · **HBase**

4 관계형 DB의 그래프 확장

쓰던 **관계형 DB**에 그래프 기능을 추가

- PostgreSQL · Apache AGE
- Oracle Graph
- SQL Server 그래프 테이블

VI · 그래프 DB의 종류와 특성

그래프 DB의 종류 ② 질의 언어와 배포 형태

</> 질의 언어 — 세 갈래

Cypher / GQL

GQL = ISO 국제 표준

Neo4j · Memgraph · Apache AGE

Gremlin

여러 제품이 공통 지원하는 탐색 언어

JanusGraph · Amazon Neptune · Cosmos DB

SPARQL

RDF 저장소의 W3C 표준

GraphDB · Stardog

배포 형태 — 세 가지

단일 서버형

운영이 단순 · 중소 규모

→

분산 클러스터형

수십억 관계 규모로 확장

→

클라우드 관리형

Neo4j AuraDB · Amazon Neptune · Cosmos DB

← 운영 단순

설치·운영 부담 ↓ →

VI · 그래프 DB의 종류와 특성

종류별 특성 비교

종류	대표 제품	강점	유의점
네이티브	Neo4j · TigerGraph	+ 탐색 성능 · 개발 생산성	– 전용 시스템 운영
멀티모델	ArangoDB · Cosmos DB	+ 문서 + 그래프 한 시스템	– 깊은 탐색은 네이티브보다 느림
분산 저장소 계층	JanusGraph	+ 대규모 확장	– 구성 요소 多 · 운영 복잡
RDF 트리플 스토어	GraphDB · Stardog	+ 표준 어휘 · 추론	– 낯선 모델링 · 튜닝 어려움
관계형 확장	Apache AGE · Oracle	+ 기존 DB 그대로 활용	– 깊은 탐색에 한계

선택 기준 질의 깊이 데이터 규모 기존 인프라 팀 역량

✓ 이 강의의 실습 → **Neo4j** (네이티브)

VII · NEO4J 개요와 위상

Neo4j 개요와 위상

Neo4j

ACID 트랜잭션 지원
네이티브 그래프 데이터베이스 관리 시스템

DB-Engines 순위 오랜 기간 1위



그래프 DBMS 부문 · 사실상의 표준 위치 (모식도)

Cypher 질의 언어 제공

Cypher → **GQL 국제 표준**의 기반

공식 드라이버

Python · Java · JavaScript · .NET · Go · 등

다양한 언어에서 손쉽게 접속

애플리케이션 ↔ Neo4j 연결이 쉽다

그래프 DB 입문 = Neo4j로 시작

VII · NEO4J 개요와 위상

Neo4j의 다섯 가지 특징 ①

1 **네이티브 그래프 저장**
노드 · 관계 · 속성을 **전용 저장 구조**로 관리
관계 탐색에 최적화

2 **ACID 트랜잭션**
정합성이 중요한 업무에 적용 가능
은행 공공

3 **유연한 스키마**
라벨 기반의 선택적 스키마 → 모델 변경에 유연하게 대응
:사람 :모임

4 · 고성능 탐색 5 · 풍부한 생태계 → 다음 장면

VII · NEO4J 개요와 위상

Neo4j의 다섯 가지 특징 ②

1 · 네이티브 저장 | 2 · ACID | 3 · 유연한 스키마

4 고성능 탐색

Index-free Adjacency로 다단계 관계 질의를 빠르게 처리

이웃만 따라간다

5 풍부한 생태계

APOC 프로시저 | 벡터 인덱스 | 시각화 도구 | 각종 커넥터

이 다섯 가지가 Neo4j를 사실상의 표준으로 만든 힘

네이티브 그래프 저장 | ACID 트랜잭션 | 유연한 스키마 | 고성능 탐색 | 풍부한 생태계

VIII · NEO4J 아키텍처

Neo4j 아키텍처 — 네 개의 계층

애플리케이션 계층
Application

- 드라이버
- Browser
- Bolt 프로토콜로 접속

질의 계층
Query

- Cypher 파싱
- 실행 계획 수립
- 통계 기반

엔진 계층
Engine

- 트랜잭션
- 인덱스
- 페이지 캐시

저장 계층
Storage

- 노드 저장소
- 관계 저장소
- 속성 저장소
- 고정 크기 레코드

성능 튜닝의 두 축
어디를 보면 빨라지는가

질의 계층
실행 계획
Cypher가 어떤 순서로 탐색하는가

엔진 계층
페이지 캐시
데이터가 메모리에 올라와 있는가

[그림 1-3] Neo4j 계층 구조

IX · 에디션과 배포 형태

에디션과 배포 형태 ① 라이선스 · DB · 클러스터

	Community Edition 오픈소스	Enterprise Edition 상용	AuraDB 클라우드
라이선스	**오픈소스** GPLv3	**상용 라이선스** 유료 계약	**구독형 서비스** 플랜 기반
DB 개수	**단일 데이터베이스** DB 1개	**다중 데이터베이스** 여러 DB 운영	**플랜에 따라 상이** 요금제별
클러스터	**미지원** 단일 서버	**지원** 고가용성 · 읽기 확장	**자동 제공** 서비스가 관리

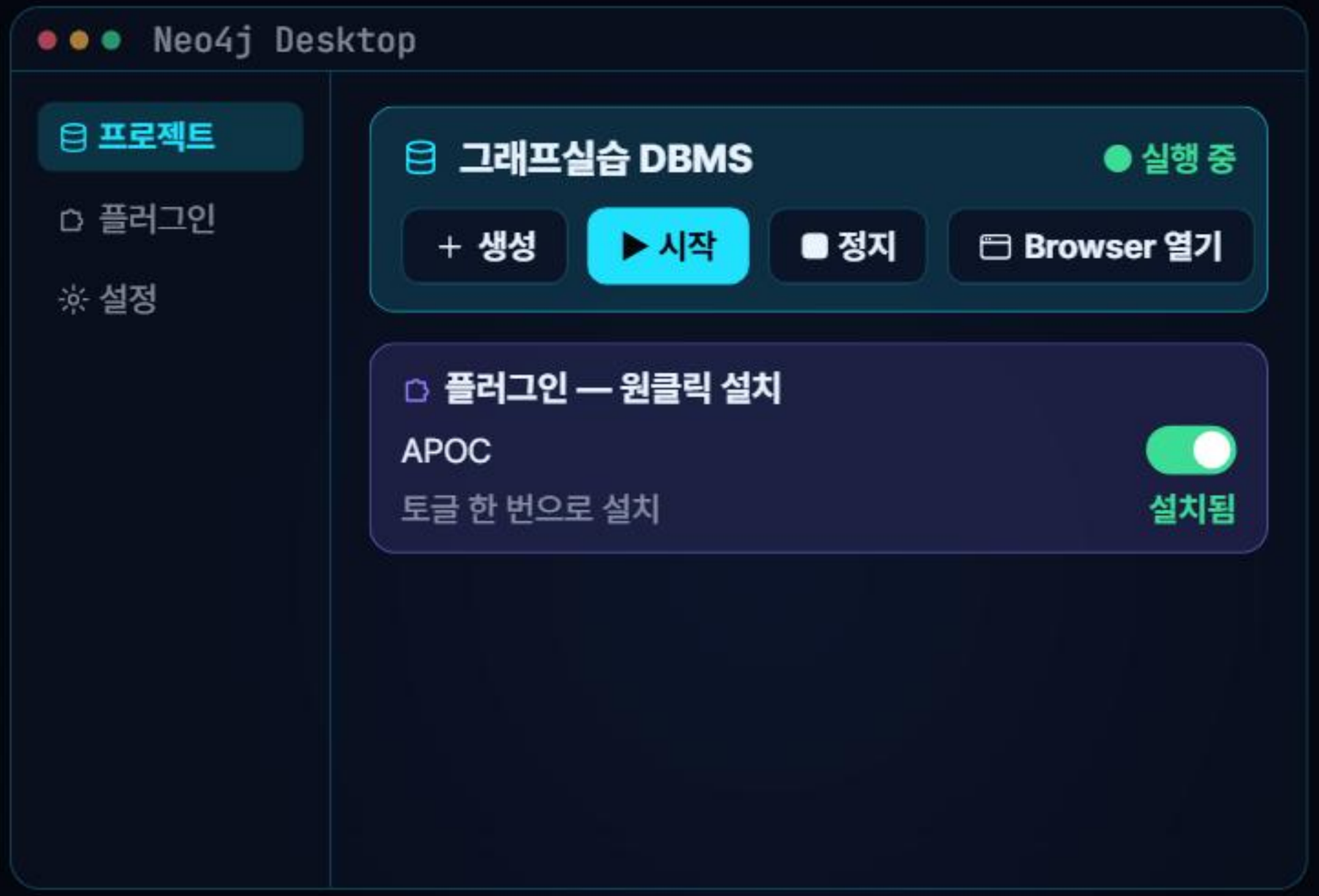
에디션과 배포 형태 ② 보안 · 적합 용도

	Community Edition 오픈소스	Enterprise Edition 상용	AuraDB 클라우드
보안	**기본 인증** 사용자명 · 비밀번호	**역할 기반 접근제어** 세밀한 권한 설정	**관리형 보안** 서비스가 보안 운영
적합 용도	**학습 · PoC · 소규모 서비스** 개인 실습, 개념 검증	**엔터프라이즈 운영** 대규모 · 고가용성 업무	**인프라 운영 부담 최소화** 설치 · 운영 없이 바로 사용

교육 · 실습에는 **Community Edition** 또는 **AuraDB Free** 계층으로 충분 우리 수업

X · 실습 환경 구축

실습 환경 ① Neo4j Desktop



방법 1 — 권장

GUI로 DBMS를 생성 · 시작 · 정지
플러그인은 **원클릭 설치**

학습 · 개인 실습에 가장 간편

처음 배우는 사람에게 적합

버튼 몇 번이면 끝

생성 → 시작 → 정지

설치 과정에서 막힐 일이 거의 없다

X · 실습 환경 구축

실습 환경 ② Docker

방법 2 — 재현 가능한 환경

terminal

```
docker run -d --name 그래프실습 \
  -p 7474:7474 -p 7687:7687 \
  -e NEO4J_AUTH=neo4j/실습비밀번호123 \
  -e NEO4J_PLUGINS='["apoc"]' \
  -v $PWD/data:/data \
  -v $PWD/import:/var/lib/neo4j/import \
  neo4j:latest
```

⚠ 학교 **내부망**에서 실습한다면
Docker 이미지를 미리 받아 둘 수 있는지 확인

포트

7474 · HTTP
Browser 접속

7687 · Bolt
드라이버 접속

인증 · 플러그인

사용자명 **neo4j** + 비밀번호를 환경 변수로
플러그인에 **APOC** 지정

볼륨

data · import 폴더 연결
컨테이너를 지워도 **데이터가 남는다**

X · 실습 환경 구축

실습 환경 ③ AuraDB Free

브라우저만 → AuraDB Free (관리형 클라우드) → 즉시 시작 ~~설치 과정~~

⚠ 꼭 기억할 점 — 최초 생성 시 발급되는 비밀번호

password • • • • • • • • • • • • •

⊗ 나중에 재확인 불가

발급 즉시 반드시 별도로 안전하게 보관

어떤 방법을 쓰든

- Neo4j Desktop
- Docker
- AuraDB Free

↓ 접속 후 환경 확인 명령은 같다

X · 실습 환경 구축

접속 확인 명령

neo4j$

```
:server status                  // 접속 상태 확인
:schema                         // 인덱스 · 제약조건 조회
CALL db.labels();               // 라벨 목록
CALL db.relationshipTypes();    // 관계 타입 목록
CALL dbms.components();         // 버전 정보
```

접속이 되었는지, 무엇이 들어 있는지, 어떤 버전인지를
다섯 줄로 확인한다

Browser

라벨

사람 모임

관계 타입

소속

속성 키

이름 나이



클릭하면 즉시 시각화

XI · 첫 한글 데이터 만들기

첫 한글 데이터 ① CREATE

Cypher는 **한글 라벨 · 관계 타입 · 속성 키 · 변수명**을 그대로 사용

create.cypher

```
// 노드 3개와 관계 2개를 생성한다
CREATE (김민수:사람 {이름:'김민수', 나이:35, 가입연도:2018})
CREATE (이서연:사람 {이름:'이서연', 나이:29, 가입연도:2021})
CREATE (요리동호회:모임 {이름:'요리동호회', 장소:'강남'})
CREATE (김민수)-[:소속 {가입일:date('2022-03-01')}]->(요리동호회)
CREATE (이서연)-[:소속 {가입일:date('2023-07-01')}]->(요리동호회);
```

5 줄의 CREATE → **3** 노드 + **2** 관계

만들어지는 그래프



첫 한글 데이터 ② MATCH 확인

query.cypher

```
// 확인 질의
MATCH (사람:사람)-[:소속]->(모임:모임 {이름:'요리동호회'})
RETURN 사람.이름 AS 이름, 사람.나이 AS 나이
ORDER BY 사람.가입연도;
```

결과

이름	나이	(가입연도)
김민수	35	2018
이서연	29	2021

```
(회원:`요리 동호회`)
```

공백 · 특수문자가 든 이름은
백틱(`)으로 감싼다

패턴 : 사람 → :소속 → 모임(요리동호회)
찾을 모양을 그대로 그린다

한글 식별자 학습 · 사내 프로젝트에 편리

⚠ 외부 협업 코드 영문 병기 권장

XII · 적합한 경우와 아닌 경우

그래프 DB가 적합한 경우

	그래프 DB가 유리	관계형 DB가 유리
질의 성격	다단계 관계 탐색 · 경로 추적	대량 집계 · 정형 리포트
데이터 구조	관계가 자주 바뀌고 다양함	구조가 고정적이고 안정적
탐색 깊이	3단계 이상	1~2단계
대표 업무	자금 흐름 추적 · 권한 경로 · 지식그래프	회계 마감 · 정산 · 통계 집계

XII · 적합한 경우와 아닌 경우

대체가 아니라 병용

~~대체 (Replace)~~ → **병용 (Combine)** 실무의 최적해

관계형 DB

정형 집계

회계 마감 · 정산 · 통계 · 리포트

함께 쓴다

그래프 DB

관계 탐색

경로 추적 · 권한 경로 · 지식그래프

"전사 DB를 **전부 그래프로 바꾸자**"

이런 결론으로 가면 안 된다

그래프가 **꼭 필요한 업무**만 골라
적용 범위를 좁게 잡는 것이 도입 성공의 조건

XIII · 도입 시 고려할 리스크

도입 시 리스크 ①

	⚠ 리스크	✓ 대응 방향
모델 설계	질의 정의 없이 모델을 먼저 그려 **재작업**	**답해야 할 질문 목록**에서 출발
학습 곡선	**SQL 사고방식**으로 Cypher 작성	**패턴 사고** 훈련 · 표준 예제 확보
데이터 동기화	원천 시스템 변경이 그래프에 **미반영**	**증분 동기화** 절차를 초기에 설계

XIII · 도입 시 고려할 리스크

도입 시 리스크 ②

	⚠ 리스크		✓ 대응 방향
운영 인력	⚠ 그래프 **운영 경험 부족**	→	✓ **관리형 서비스** 활용 · 운영 문서화
라이선스	⚠ 기능이 **에디션에 따라 상이**	→	✓ 필요 기능을 **사전에 에디션과 대조**

◎ 다섯 가지 모두 기술 자체보다 **준비와 절차**의 문제

모델 설계 | 학습 곡선 | 데이터 동기화 | 운영 인력 | 라이선스

정리

정리 ① 개념 되짚기

오늘 배운 핵심 세 가지

패러다임의 변화

데이터의 가치는 **개체 → 연결**, 질의는 **집계형 → 탐색형**

개체 → 연결

관계형 DB의 한계

관계를 **외래키**로만 표현 → 깊은 탐색에서 **JOIN 중첩**, 비용 급증

FK → JOIN × N

그래프 DB

관계를 **1급 시민**으로 저장 · **Index-free Adjacency**로 이웃만 따라가며 탐색

노드 — 관계 — 노드

정리

정리 ② Neo4j와 실습, 도입 원칙

Neo4j

LPG 모델의 네이티브 그래프 DB · **사실상의 표준**

- ACID
- Cypher → GQL
- Index-free Adjacency
- 풍부한 생태계

실습 환경

셋 중 하나로 시작한다

- Neo4j Desktop
- Docker
- AuraDB Free

첫 한글 데이터

한글 식별자로 CREATE · MATCH 바로 가능

외부 협업 코드는 영문 병기 권장

```
(김민수:사람)-[:소속]->(요리동호회:모임)
```

도입 원칙

관계형의 **대체가 아니라 병용**

적용 범위를 좁게 잡는 것이 성공 조건

- 정형 집계 = 관계형
- 관계 탐색 = 그래프

✦ **그래프 데이터베이스 개요와 Neo4j — 다음 강의에서 뵙겠습니다**